

Assignment – 8

1. Create a ThemeContext using React's Context API and useReducer to toggle between 'light' and 'dark' modes in a simple app, then display the current theme in a header.
2. Build a Zomato-style restaurant favorites feature: set up two nested context providers (UserContext and FavoritesContext) and allow the user to add/remove restaurant IDs to their favorites list using reducer actions.
3. Refactor a context value provider to use React.memo or useMemo so that a child component (like a Playlist component in a Spotify-style app) does not re-render unnecessarily when unrelated context state changes.

Hint: Use useMemo to memoize the context value object.
4. Given a simplified Instagram-like app with AuthContext and a reducer handling login/logout actions, add a new reducer action to update the user's display name, and update the context so that components show the new name immediately after dispatch.
5. Use ChatGPT to generate a reducer function for managing a shopping cart context (add, remove, clear cart) and integrate it into a CartContext provider in your React app.

Hint: Paste your prompt and the generated reducer code in your submission.

Answer:- 1. ThemeContext with useReducer

```
import { createContext, useReducer } from "react";

export const ThemeContext = createContext();

const initialState = {
  theme: "light",
};

function themeReducer(state, action) {
  switch (action.type) {
    case "TOGGLE_THEME":
      return {
        theme:
          state.theme === "light"
            ? "dark"
            : "light",
      };
    default:
      return state;
  }
}
```

```

}

export const ThemeProvider = ({ children }) => {
  const [state, dispatch] = useReducer(
    themeReducer,
    initialState
  );

  return (
    <ThemeContext.Provider
      value={{ state, dispatch }}
    >
      {children}
    </ThemeContext.Provider>
  );
};

```

Header.jsx

```

import { useContext } from "react";
import { ThemeContext } from "../ThemeContext";

const Header = () => {
  const { state, dispatch } =
    useContext(ThemeContext);

  return (
    <>
      <h1>Current Theme: {state.theme}</h1>

      <button
        onClick={() =>
          dispatch({
            type: "TOGGLE_THEME",
          })
        }
      >
        Toggle Theme
      </button>
    </>
  );
};

export default Header;

```

2. Zomato Favorites Feature

```

import { createContext } from "react";

```

```

export const UserContext = createContext();

export const UserProvider = ({
  children,
}) => {
  const user = {
    id: 1,
    name: "Nena",
  };

  return (
    <UserContext.Provider value={user}>
      {children}
    </UserContext.Provider>
  );
};

```

FavoritesContext.jsx

```

import {
  createContext,
  useReducer,
} from "react";

export const FavoritesContext =
  createContext();

function reducer(state, action) {
  switch (action.type) {
    case "ADD_FAVORITE":
      return [
        ...state,
        action.payload,
      ];

    case "REMOVE_FAVORITE":
      return state.filter(
        (id) => id !== action.payload
      );

    default:
      return state;
  }
}

export const FavoritesProvider = ({
  children,

```

```

})) => {
  const [favorites, dispatch] =
    useReducer(reducer, []);

  return (
    <FavoritesContext.Provider
      value={{
        favorites,
        dispatch,
      }}
    >
      {children}
    </FavoritesContext.Provider>
  );
};

```

RestaurantCard.jsx

```

import { useContext } from "react";
import { FavoritesContext } from "../FavoritesContext";

const RestaurantCard = ({ id }) => {
  const { favorites, dispatch } =
    useContext(FavoritesContext);

  const isFavorite =
    favorites.includes(id);

  return (
    <>
      <h3>Restaurant {id}</h3>

      <button
        onClick={() =>
          dispatch({
            type: isFavorite
              ? "REMOVE_FAVORITE"
              : "ADD_FAVORITE",
            payload: id,
          })
        }
      >
        {isFavorite
          ? "Remove Favorite"
          : "Add Favorite"}
      </button>
    </>
  );
};

```

```
};  
  
export default RestaurantCard;
```

3. Prevent Re-renders using useMemo

Before

```
<PlaylistContext.Provider  
  value={{  
    songs,  
    currentSong,  
    setCurrentSong  
  }}  
>
```

After:-

```
import { useMemo } from "react";  
  
const value = useMemo(  
  () => ({  
    songs,  
    currentSong,  
    setCurrentSong,  
  }),  
  [songs, currentSong]  
);  
  
return (  
  <PlaylistContext.Provider  
    value={value}  
  >  
    {children}  
  </PlaylistContext.Provider>  
)
```

4. Instagram AuthContext

```
function authReducer(  
  state,  
  action  
) {  
  switch (action.type) {  
    case "LOGIN":  
      return {  
        ...state,  
        isLoggedIn: true,  
      }  
    }  
  }  
}
```

```
        user: action.payload,  
    };  
  
    case "LOGOUT":  
        return {  
            isLoggedIn: false,  
            user: null,  
        };  
  
    case "UPDATE_NAME":  
        return {  
            ...state,  
            user: {  
                ...state.user,  
                displayName:  
                    action.payload,  
            },  
        };  
  
    default:  
        return state;  
    }  
}
```